# Assessment Task: AI/ML Engineer

# Candidate Details

# Name:

# Email:

# Phone:

**Overview**

As part of this assessment, you are required to complete two practical AI/ML tasks. The first task evaluates your ability to build and evaluate a classical machine learning solution using real-world industrial data. The second task evaluates your ability to build a small Retrieval-Augmented Generation (RAG) application using open-source AI tools.

The primary focus of the assessment is problem understanding, data preparation, model selection, evaluation, software engineering, and the ability to explain technical decisions. All tasks can be completed using free and open-source resources. Paid APIs are not required.

## Task 1 – Machine Learning

**Predictive Maintenance – Machine Failure Prediction**

Build a machine learning system that predicts whether an industrial machine is likely to experience a failure based on operating and sensor measurements.

**Dataset:** UCI AI4I 2020 Predictive Maintenance Dataset

**ML Task Requirements**

The solution should:

1. Perform exploratory data analysis and understand the available features and target variable.
2. Identify missing values, outliers, class imbalance, and other data quality issues.
3. Perform appropriate preprocessing and feature engineering.
4. Split the data into training and test sets using a methodology appropriate for the problem.
5. Train at least three different classification models, such as Logistic Regression, Random Forest, SVM, XGBoost, or LightGBM.
6. Select appropriate evaluation metrics and explain why they are suitable for machine-failure prediction.
7. Compare the models and perform reasonable hyperparameter tuning.
8. Analyze feature importance or another suitable model explainability method.
9. Perform error analysis and discuss false positives and false negatives.
10. Save the final trained model and provide a way to make predictions on new input data.

**Important ML Requirement**

Do not evaluate the model using accuracy alone. Since machine failure prediction can involve class imbalance and different costs for false positives and false negatives, explain the chosen metrics and their business significance.

**Suggested ML Technology Stack**

- Python
- Pandas / NumPy
- scikit-learn
- Matplotlib / Plotly
- Optional: SHAP
- Optional: Streamlit for a simple prediction interface

**Expected ML Submission**

- README.md with setup and execution instructions. Clearly explain the basis for selecting the models of your choice, benchmarking details between different models, and your understanding on why certain model performs better than the other.
- Jupyter notebook containing EDA and experimentation
- Clean Python source code for preprocessing, training, evaluation, and prediction
- requirements.txt
- Saved model artifact
- Evaluation report describing model selection, metrics, results, and limitations
- Optional Streamlit application

# Paste your solution GitHub Repo Link here:

## Task 2 – Artificial Intelligence

**Technical Document Question-Answering System using RAG**

Build an AI application that allows a user to ask questions about a collection of technical documents. The application should retrieve relevant information from the documents and use an open-source language model to generate an answer grounded in the retrieved information.

**Dataset / Open-Source Resources**

**BEIR information-retrieval benchmark**

**AI Task Requirements**

The system should implement the following flow:

**Documents → Text Extraction → Chunking → Embeddings → Vector Search → Retrieval → LLM → Answer + Sources**

11. Ingest PDF or text documents.
12. Extract the document text while retaining useful metadata such as document name and page number.
13. Split the documents into meaningful chunks and explain the selected chunk size and overlap.
14. Generate embeddings using a free open-source embedding model such as sentence-transformers/all-MiniLM-L6-v2 or BAAI/bge-small-en-v1.5.
15. Store embeddings using a free vector-search solution such as FAISS, Chroma, or Qdrant.
16. Implement semantic retrieval and return the most relevant document chunks for a user query.

17. Use a locally runnable open-source language model such as Qwen, Llama, Mistral, or Gemma to generate the final response. (Use the most light-weight LLM you can find out. You will not be evaluated based on the quality of answers generated by the LLM).
18. Ensure that the answer is grounded in the retrieved context and does not unnecessarily invent information.
19. Display document and page-level source references for the generated answer.
20. Handle questions for which the provided documents do not contain sufficient information.

**Important AI Requirement**

The system must not blindly generate an answer when the required information is unavailable in the retrieved documents. It should clearly indicate when the provided knowledge base does not contain sufficient information.

**Suggested AI Technology Stack**

- Python
- Sentence Transformers / Hugging Face
- FAISS / Chroma / Qdrant
- Qwen / Llama / Mistral / Gemma
- Ollama, llama.cpp, or Hugging Face Transformers for local inference
- PyMuPDF or another open-source PDF extraction library
- Streamlit for the user interface
- Optional: FastAPI for a backend API

**Explain the following in README**

21. What is the main objective of the selected document?
22. What methodology was used?
23. What are the major experimental or technical parameters?
24. What conclusions did the authors reach?
25. Compare the methodology used in two documents.
26. What evidence supports a particular conclusion?
27. Ask a question whose answer is not contained in the knowledge base and demonstrate how the system responds.

**Expected AI Submission**

- README.md with setup and execution instructions. Write one paragraph on “How can we obtain the best answers from a very light-weight LLM? Think in terms of documents pre-processing”.
- Source code for ingestion, chunking, embeddings, retrieval, generation, and the RAG pipeline
- requirements.txt
- Evaluation questions and example outputs
- Short evaluation report discussing retrieval quality, answer quality, limitations, and hallucination handling
- Working UI or CLI demonstration
- Optional Docker configuration

# Paste your solution GitHub Repo Link here:

## General Technical Requirements

- Use Python and maintain clean, modular code.
- Use Git for version control.
- Do not use paid APIs or proprietary services that require payment.
- Open-source models and libraries are permitted.
- Clearly document installation, dependencies, execution steps, and design decisions.
- The candidate should be able to explain all major parts of the submitted implementation.

## Evaluation Criteria

The assessment will be evaluated approximately as follows:

| Area | ML | AI |
|---|---|---|
| Problem understanding / EDA | 15 | 10 |
| Data / document preprocessing | 10 | 10 |
| Model / retrieval architecture | 15 | 20 |
| Evaluation methodology | 20 | 15 |
| Feature engineering / prompting | 10 | 10 |
| Error / hallucination analysis | 10 | 15 |
| Code quality & reproducibility | 10 | 10 |
| Demo / usability | 10 | 10 |
| Total | 100 | 100 |

## Submission

Fill this google form to submit your assessment:

https://docs.google.com/forms/d/e/1FAIpQLSfKoihFferp1Q0hYDEEnMwLKLQfQNteX2vzgcbrkiM7LmgTSA/viewform?usp=publish-editor

## Bonus

- Dockerize the application.
- Build a FastAPI endpoint for inference.
- Add automated tests.
- Add model/version tracking.
- Add SHAP or another explainability method for the ML task.
- Add retrieval evaluation such as Recall@K or MRR for the AI task.
- Add a simple Streamlit dashboard.
- Demonstrate a local-only deployment with no paid external API.